Equation d'ALLEN-CAHN

présenté par

Cléa MARIN

Raphael MARQUES ARAUJO

Projet EDP 2024/2025

Contents

1 Introduction **3**
1.1 Enoncé de l'équation,hypothèses et conditions aux limites 3

2 Résolution de l'équation par la méthode des différences finies **3**
2.1 Schéma explicite 3
2.2 Schéma implicite 5
2.3 Consistance et ordre des schémas 6
2.3.1 Schéma explicite 6
2.3.2 Schéma implicite 7
2.4 condition de Courant-Friedrichs-Lewy 7

3 Mis en oeuvre informatique **7**
3.1 Simulation de la méthode explicite en 1D avec une solution manufacturée . . 7
3.2 3.2 Simulation de la méthode implicite en 1D 8
3.3 Simulation de l'équation en 2D 9
3.4 3.3 Simulation d'un cas connu : Transition de phase sur un front vertical (1D et 2D) 10
3.5 3.4 Ajout d'un terme de transport 11

1 Introduction

L'équation d'Allen-Cahn est l'equation aux dérivés partielles qui decrit la separation dans des alliages binaires. On peut l'utiliser par exemple dans le cadre de la physique des matériaux où on veut pouvoir étudier la répartition des structures des composants de l'alliage suite à un processus de chauffage ou de refroidissement rapide. Plus précisément, cette équation nous donne une organisation spatiale de l'alliage.

1.1 Enoncé de l'équation,hypothèses et conditions aux limites

L'équation de Allen-Cahn est décrite par :

$$\frac{\partial u}{\partial t} - \varepsilon \Delta u + f(u) = 0, \quad (x,t) \in \Omega \times (0,T) \tag{1}$$

$$u(x,0) = u_0(x), \quad x \in \Omega \tag{2}$$

$$u(x,t) = 0, \quad x \in \partial\Omega \tag{3}$$

où $\partial\Omega$ désigne la frontière de Ω.

Avec :

$$f(u) = F'(u), \quad \text{et} \quad F(u) = \frac{1}{4}(u^2-1)^2 \tag{4}$$

u(x,t)	phase du matériau qu 'on considère
ϵ	paramètre positif dépendant du matériau
Ω	domaine spatiale (ouvert)

Table 1: Table des variables et domaines de l'équation

2 Résolution de l'équation par la méthode des différences finies

2.1 Schéma explicite

Avant de commencer la résolution, nous devons émettre un certain nombre d'hypothèses sur les fonctions qu'on traite afin de garantir l'existence et le caractère bien défini de ces équations. Tout d'abord on suppose $u \in C^2(\Omega)$ tout simplement car on traite avec des dérivées partrielles secondes. On suppose par ailleurs dans un premier temps que le problèmes est unidimensionnelle de sorte à ce que $\frac{\partial^2 u}{\partial y^2} = \frac{\partial^2 u}{\partial z^2} = 0$ En outre justifions que F est dérivable et donnons une expérience explicite de F'. En effet F est dérivable en tant que composé de

$$g : x \mapsto \frac{1}{4}(x^2-1)^2$$

qui est dérivable en tant que polynôme et u $\in C^2(\Omega)$. Donc il vient que F est dérivable et

$$F' : u \mapsto u(u^2 - 1)$$

On construit un maillage de notre espace spatio-temporel, on échantillonne N points sur l'intervalle Ω à intervalle régulier Δx, et on découpe le temps en intervalles réguliers Δt.

On désigne par u_i^n l'approximation de la fonction u au point (x_i, t_n), tel que :

$$x_i = i \times \Delta x \quad \text{et} \quad t_n = n \times \Delta t \quad \text{avec i et n } \in N \quad i \in [|0, N|]$$

On rappelle que dans le cas unidimensionnelle on obtient l'équation suivante :

$$\frac{\partial u}{\partial t} - \varepsilon \frac{\partial^2 u}{\partial x^2} + f(u) = 0 \tag{1}$$

En appliquant la méthode des différences finies qui repose sur une approximation des dérivées partielles à l'aide du développement Taylor suivi d'une discrétisation il vient que :

$$\frac{\partial u}{\partial t}(x_i, t^n) \approx \frac{u_i^{n+1} - u_i^n}{\Delta t} \tag{2}$$

$$\frac{\partial^2 u}{\partial x^2}(x_i, t^n) \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} \tag{3}$$

$$\frac{\partial u}{\partial t}(x_i, t^n)\Big|_i^n = \frac{u_i^{n+1} - u_i^n}{\Delta t} \tag{4}$$

$$\frac{\partial^2 u}{\partial x^2}(x_i, t^n)\Big|_i^n = \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} \tag{5}$$

Donc en injectant dans l'équation dans (1) il vient que :

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \varepsilon \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} - f(u_i^n). \tag{6}$$

En posant $\alpha = \frac{\epsilon \Delta t}{(\Delta x)^2}$, on obtient :

$$u_i^{n+1} = u_i^n(1 - 2\alpha) + \alpha u_{i+1}^n + \alpha u_{i-1}^n - \Delta t \left((u_i^n)^3 - u_i^n\right). \tag{7}$$

On réitère ce schéma N fois et il vient que :

$$\begin{aligned}
u_1^{n+1} &= u_1^n + \alpha(u_2^n - 2u_1^n + u_0^n) - \Delta t \left((u_1^n)^3 - u_1^n\right) \\
u_2^{n+1} &= u_2^n + \alpha(u_3^n - 2u_2^n + u_1^n) - \Delta t \left((u_2^n)^3 - u_2^n\right) \\
&\vdots \\
u_{N-1}^{n+1} &= u_{N-1}^n + \alpha(u_N^n - 2u_{N-1}^n + u_{N-2}^n) - \Delta t \left((u_{N-1}^n)^3 - u_{N-1}^n\right)
\end{aligned}$$

Cependant d'après les conditions limites décrites dans (3) on en déduit que $u_0^n = u_{N-1}^n = 0$ donc

$$
\begin{aligned}
u_1^{n+1} &= u_1^n + \alpha(u_2^n - 2u_1^n) - \Delta t\left((u_1^n)^3 - u_1^n\right)\\
u_2^{n+1} &= u_2^n + \alpha(u_3^n - 2u_2^n + u_1^n) - \Delta t\left((u_2^n)^3 - u_2^n\right)\\
&\vdots\\
u_{N-1}^{n+1} &= u_{N-1}^n + \alpha(-2u_{N-1}^n + u_{N-2}^n) - \Delta t\left((u_{N-1}^n)^3 - u_{N-1}^n\right)
\end{aligned}
$$

On considère le vecteur inconnu :

$$
u^n = \begin{bmatrix} u_1^n \\ u_2^n \\ \vdots \\ u_{N-2}^n \\ u_{N-1}^n \end{bmatrix} \in R^{N-1}
$$

Le schéma explicite associé à l'équation d'Allen–Cahn avec conditions de Dirichlet homogènes s'écrit, pour les points intérieurs $i = 1, \ldots, N-1$, sous forme matricielle :

$$
u^{n+1} = \begin{bmatrix} 1-2\alpha & \alpha & 0 & \cdots & 0 \\ \alpha & 1-2\alpha & \alpha & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \alpha & 1-2\alpha & \alpha \\ 0 & \cdots & 0 & \alpha & 1-2\alpha \end{bmatrix} u^n - \Delta t\left((u^n)^{\circ 3} - u^n\right)
$$

où $(u^n)^{\circ 3} = \begin{bmatrix} (u_1^n)^3 \\ (u_2^n)^3 \\ \vdots \\ (u_{N-2}^n)^3 \\ (u_{N-1}^n)^3 \end{bmatrix}$

2.2 Schéma implicite

De manière analogue on construit le schéma implicite, on approxime la dérivée partielle spatiale cette fois ci à l'instant n+1 et on obtient :

$$
\frac{u_i^{n+1} - u_i^n}{\Delta t} = \varepsilon \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2} - f(u_i^{n+1}). \tag{8}
$$

On regroupe et on multiplie par Δt, et on garde toujours

$$
\alpha = \frac{\varepsilon \Delta t}{(\Delta x)^2},
$$

ce qui donne :

$$
u_i^{n+1} - \alpha\left(u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}\right) + \Delta t\left((u_i^{n+1})^3 - u_i^{n+1}\right) = u_i^n. \tag{9}
$$

Il vient alors que :

$$\begin{bmatrix} 1+2\alpha & -\alpha & 0 & \cdots & 0 \\ -\alpha & 1+2\alpha & -\alpha & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -\alpha & 1+2\alpha & -\alpha \\ 0 & \cdots & 0 & -\alpha & 1+2\alpha \end{bmatrix} u^{n+1} + \Delta t \left((u^{n+1})^{\circ 3} - u^{n+1} \right) = u^n \tag{10}$$

Il reste donc à résoudre cette équation matricielle qui est non linéaire, on se propose donc d'utiliser la méthode de Newton qui repose sur l'unicité et l'existence de la solution de $\phi(z) = z$ si ϕ est contractante c'est à dire k-lipschitzienne sur un espace métrique complet où k<1.

2.3 Consistance et ordre des schémas

2.3.1 Schéma explicite

Par définition, l'erreur de consistance notée ϵ_j^nau noeud x_i et à l'instant t_n ou l'erreur de discrétisation locale décrit l'erreur qu'on commet quand on suppose que le schéma numérique approche la solution exacte :

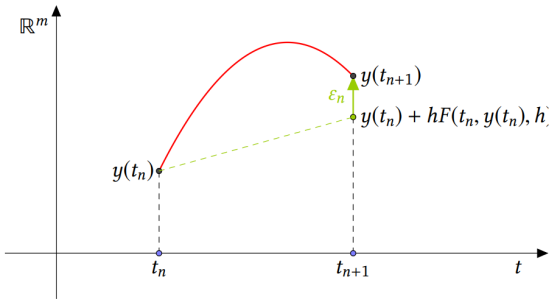

Figure 1: Erreur de consistance

Il vient que :

$$\varepsilon_i^n := \frac{u(x_i, t^{n+1}) - u(x_i, t^n)}{\Delta t} - \epsilon \frac{u(x_{i+1}, t^n) - 2u(x_i, t^n) + u(x_{i-1}, t^n)}{\Delta x^2} + u(x_i, t^n)^3 - u(x_i, t^n) \tag{11}$$

Le caractère C^∞ de u selon ses 2 variables nous permets de développer notre fonction selon Taylor :

$$u(x_i, t_{n+1}) = u(x_i, t_n) + \Delta t \frac{\partial u}{\partial t}(x_i, t_n) + \mathcal{O}((\Delta t)^2) \tag{12}$$

$$\begin{aligned} u(x_{i\pm1}, t_n) = u(x_i, t_n) \pm \Delta x \frac{\partial u}{\partial x}(x_i, t_n) + \frac{(\Delta x)^2}{2} \frac{\partial^2 u}{\partial x^2}(x_i, t_n) \\ \pm \frac{(\Delta x)^3}{6} \frac{\partial^3 u}{\partial x^3}(x_i, t_n) + \mathcal{O}((\Delta x)^4) \end{aligned} \tag{13}$$

Donc en injectant ces développements dans l'erreur de troncature on a que :

Maintenant, nous pouvons injecter ces développements dans l'expression de l'erreur de consistance.

$$\varepsilon_i^n = \left(\frac{\partial u}{\partial t}(x_i, t_n) - \epsilon \frac{\partial^2 u}{\partial x^2}(x_i, t_n) + u(x_i, t_n)^3 - u(x_i, t_n) \right) + \mathcal{O}(\Delta t) + \mathcal{O}(\Delta x^2)$$

lorsque Δt et Δx tendent tous les 2 vers 0. Donc notre schéma est consistant d'ordre 1 selon t et 2 selon x.

2.3.2 Schéma implicite

Nous reprenons la même méthode pour le schéma implicite et on obtient finalement:

La différence finie $\frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2}$ devient une approximation de la dérivée seconde spatiale :

$$\frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2} = \frac{\partial^2 u}{\partial x^2}(x_i, t_n) + \mathcal{O}(\Delta x^2) + \mathcal{O}(\Delta t)$$

Cela est de l'ordre $\mathcal{O}(\Delta x^2)$ pour le terme spatial.

Le terme non linéaire $f(u_i^{n+1})$ donne un développement de Taylor autour de $u(x_i, t_n)$:

$$f(u_i^{n+1}) = f(u(x_i, t_n)) + \Delta t(3u(x_i, t_n)^2 - 1)\frac{\partial u}{\partial t}(x_i, t_n) + \mathcal{O}(\Delta t^2)$$

En combinant les développements précédents, l'erreur de consistance du schéma est :

$$\varepsilon_i^n = \mathcal{O}(\Delta t) + \mathcal{O}(\Delta x^2)$$

Cela montre que le schéma est consistant d'ordre 1 en t et d'ordre 2 en x.

2.4 condition de Courant-Friedrichs-Lewy

La condition de Courant-Friedrichs-Lewy (CFL) est une condition nécessaire à la stabilité des schémas numériques explicites. Pour l'équation de diffusion de type Allen-Cahn donnée par

$$\frac{\partial u}{\partial t} - \varepsilon \Delta u + f(u) = 0,$$

la condition CFL impose que le rapport entre le pas de temps Δt et le carré du pas spatial Δx^2 soit suffisamment petit, c'est-à-dire que

$$\frac{\varepsilon \Delta t}{\Delta x^2} \leq \frac{1}{2}.$$

Cette condition garantit que l'information ne "saute" pas trop loin à chaque itération temporelle et

3 Mis en oeuvre informatique

3.1 Simulation de la méthode explicite en 1D avec une solution manufacturée

Nous implémentons un schéma numérique explicite en une dimension pour l'équation d'Allen-Cahn (code basé sur celui vu en TD). Pour cette première simulation, nous imposons des

conditions aux limites nulles (u = 0 aux extrémités) et une condition initiale uniforme (u = 1).

```python
while k < max_iter_time - 1 and err > TOL:
    u_new = np.copy(u[k])
    for i in range(1, Domaine - 1):
        u_new[i] = u[k][i]* (1-2*alpha) + alpha*u[k][i-1] +alpha*u[k][i+1] - delta_t*( u[k][i]**3 - u[k][i])
```

Figure 2: *Code Python de la simulation 1D*

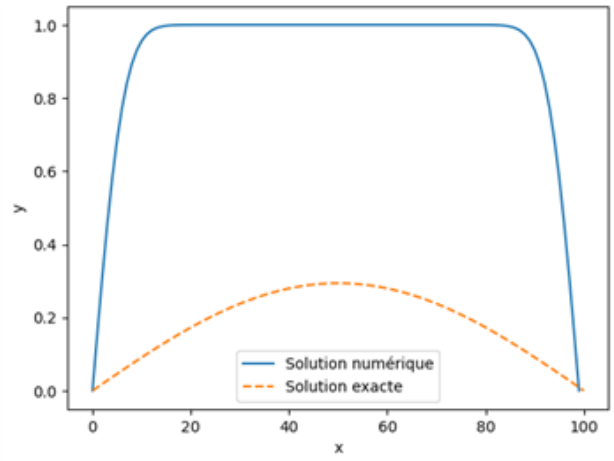

Figure 3: *solution exacte et approchée en fonction de x*

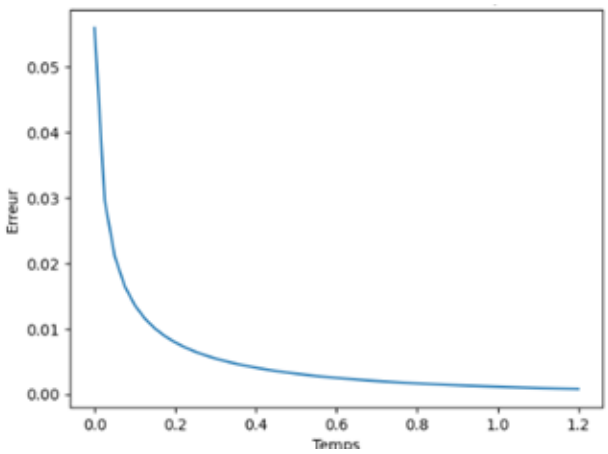

Figure 4: *évolution de l'erreur entre la solution et le schéma numérique en fonction du temps*

La courbe d'erreur montre une bonne convergence du schéma dans le temps, ce qui valide la mise en œuvre du modèle.

3.2 3.2 Simulation de la méthode implicite en 1D

Nous implémentons maintenant le schéma implicite.

```python
def cohen_allen_step(u_n, alpha, dt, max_iter=20, tol=1e-8):
    N = len(u_n)

    # Création de la matrice tridiagonale A pour la partie linéaire du Laplacien
    main_diag = (1 + 2 * alpha) * np.ones(N)
    off_diag = -alpha * np.ones(N - 1)
    A = diags([off_diag, main_diag, off_diag], [-1, 0, 1], format='csr')
    u = u_n.copy()  # Initialisation de l'inconnue u (point de départ de l'itération de Newton)

    # Boucle de Newton pour résoudre le système non linéaire
    for _ in range(max_iter):
        f_u = u**3 - u # Évaluation du terme non linéaire f(u) = u^3 - u
        df_u = 3 * u**2 - 1 # Dérivée de f(u), utilisée dans le jacobien J (matrice tangente)
        G = A @ u + dt * f_u - u_n # Construction du résidu G(u)=A@u+dt*f(u)-u_n (équation qu'on veut annuler: G(u) = 0)
        J = A + dt * diags(df_u, 0) # Construction du Jacobien de G : J(u) = A + dt * diag(f'(u))
        delta = spsolve(J, -G)  # Résolution linéaire de J * delta = -G
        u += delta  #Mise à jour de la solution : u_(k+1) = u_k + delta

        # Critère d'arrêt de Newton : si la mise à jour est suffisamment petite
        if np.linalg.norm(delta, np.inf) < tol:
            break
    return u
```

Figure 5: Code python du schéma implicite

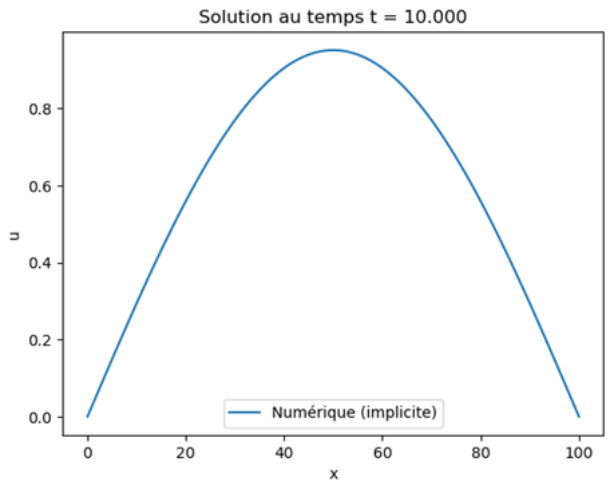

Figure 6: *solution exacte en fonction de x à t=10 s*

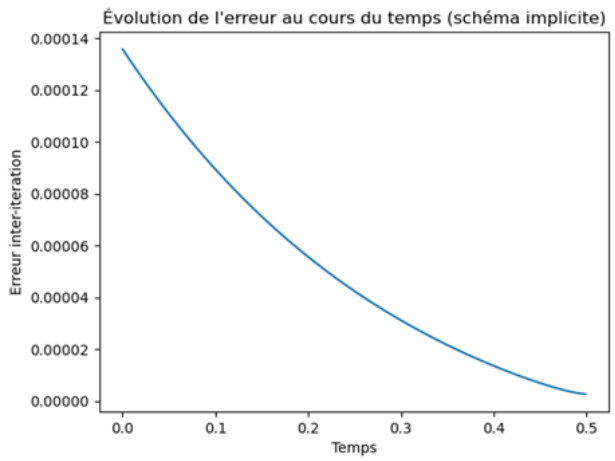

Figure 7: *évolution de l'erreur entre la solution et le schéma numérique en fonction du temps*

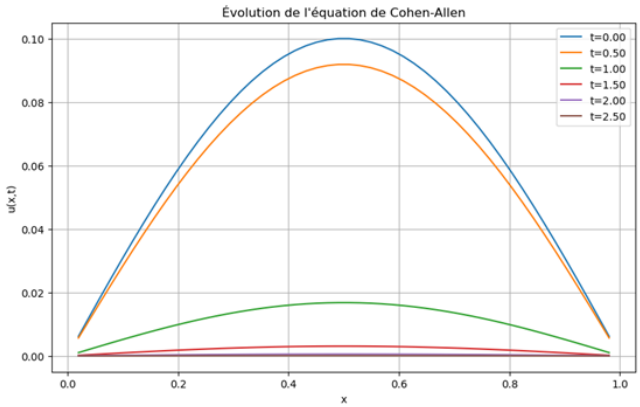

Figure 8: Graphe de la solution implicite à différents temps

On observe que le schéma implicite est stable, et présente une courbe d'erreur qui semble converger plus lentement que celle du schéma explicite mais avec un niveau beaucoup plus faible. Sur la figure 5, on voit que la solution $u(x,t)$ décroît rapidement entre t=0.5 et $t \approx 1$, avant de se stabiliser lentement vers zéro.

3.3 Simulation de l'équation en 2D

Nous avons ensuite généralisé le schéma explicite à deux dimensions. Le champ de phase est défini sur un carré, avec des conditions aux bords maintenues à zéro.

```
while k < max_iter_time-1 and err > TOL:
    u_new = np.copy(u[k])

    for i in range(1, Domaine-1):
        for j in range(1, Domaine-1):
            laplacian = (u[k][i+1][j] + u[k][i-1][j] + u[k][i][j+1] + u[k][i][j-1] - 4*u[k][i][j])
            non_linear = u[k][i][j]**3 - u[k][i][j]
            u_new[i][j] = u[k][i][j] + alpha * laplacian - delta_t * non_linear
```

Figure 9: *Code python de la simulation 2D*

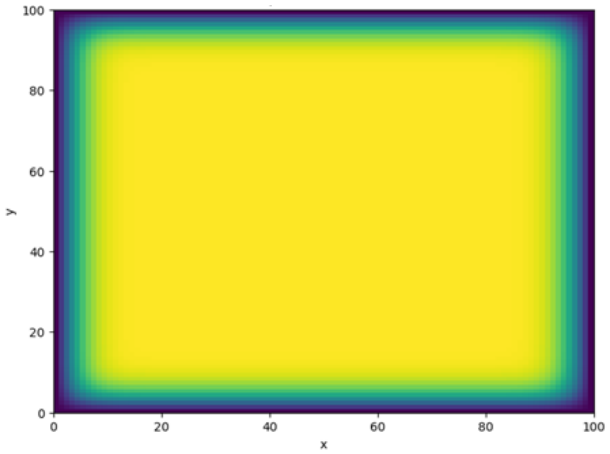

Figure 10: *Distribution de u en 2D*

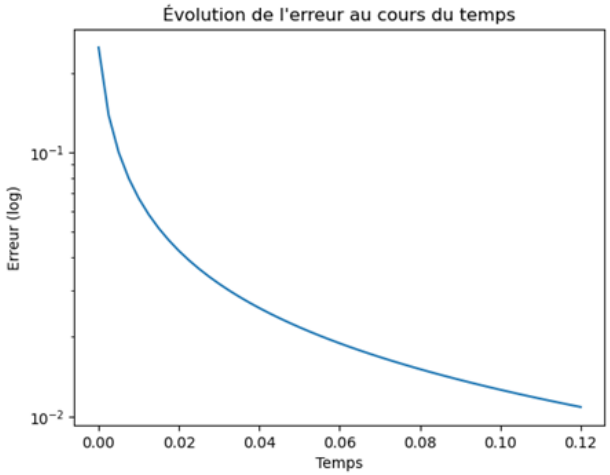

Figure 11: *Erreur en fonction du temps en 2D (norme 2)*

On constate une cohérence entre les solutions 1D et 2D : la température augmente progressivement dans la zone centrale du domaine, avant de redescendre aux bords, conformément aux conditions imposées.

3.4 3.3 Simulation d'un cas connu : Transition de phase sur un front vertical (1D et 2D)

Nous avons choisi de simuler une transition de phase dans un alliage à front vertical. Afin de réaliser cela nous initialisons le champ u avec deux valeurs distinctes : 1 à gauche et -1 à droite, et une interface à 0 au centre. Cette configuration représente deux matériaux en contact.

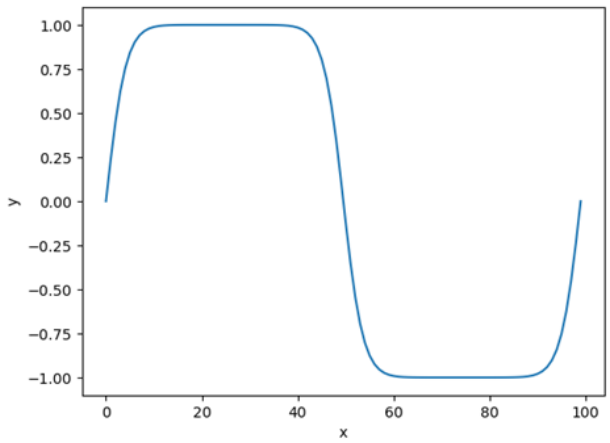

Figure 12: *Evolution du champ 1D*

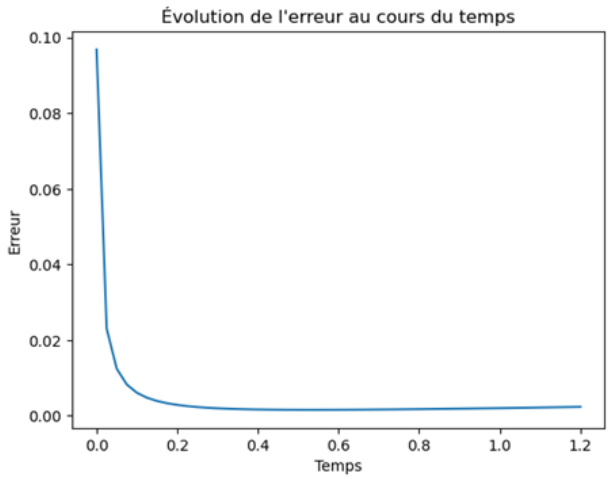

Figure 13: *Erreur en fonction du temps*

Le résultat montre nettement la diffusion entre les deux phases, avec un lissage progressif de l'interface. Nous avons ensuite étendu cette approche en deux dimensions.

```python
while k < max_iter_time - 1 and err > TOL:
    u_new = np.copy(u[k])
    for i in range(1, Domaine - 1):
        u_new[i] = u[k][i]* (1-2*alpha) + alpha*u[k][i-1] +alpha*u[k][i+1]
        - delta_t*( u[k][i]**3 - u[k][i])        -v * (u[k][i] - u[k][i-1]) / meshSize
```

Figure 16: *Code python avec ajout de la vitesse de transport*

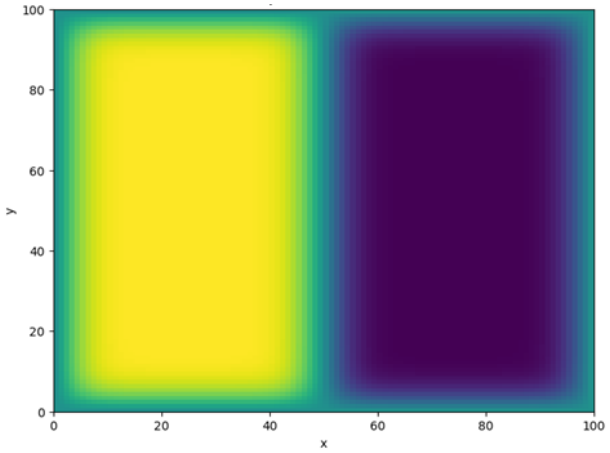

Figure 14: *Evolution du champ en 2D*

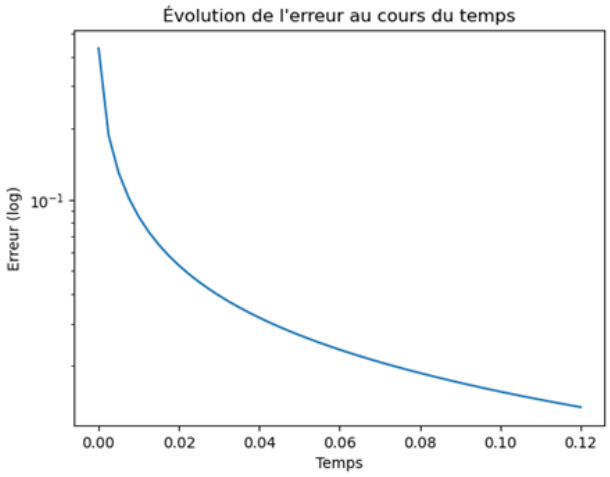

Figure 15: *Erreur en fonction du temps*

Les résultats confirment les comportements vus précédemment : la séparation nette entre les deux phases est conservée, et l'interface évolue de manière isotrope selon la dynamique de l'équation d'Allen-Cahn.

3.5 3.4 Ajout d'un terme de transport

Nous avons modifié l'équation pour introduire un terme de transport :

$$\frac{\partial u}{\partial t} = \varepsilon \Delta u - fu - v\frac{\partial u}{\partial x}$$

où v représente la vitesse de transport (ici initialisé à 0.3 m/s). Cela permet de simuler un déplacement du champ, par exemple sous l'effet d'un flux thermique ou mécanique. Le v de ce terme représente la vitesse de transport et a été initialisé à 0.3 m/s. Cela nous permet d'avoir les résultats ci-dessous.

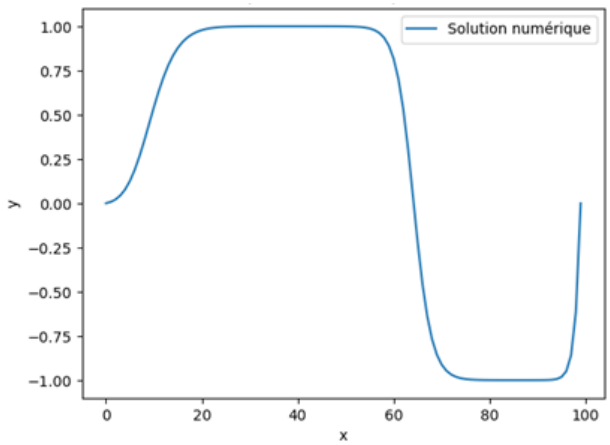

Figure 17: *Effet du transport sur l'évolution du champ de phase*

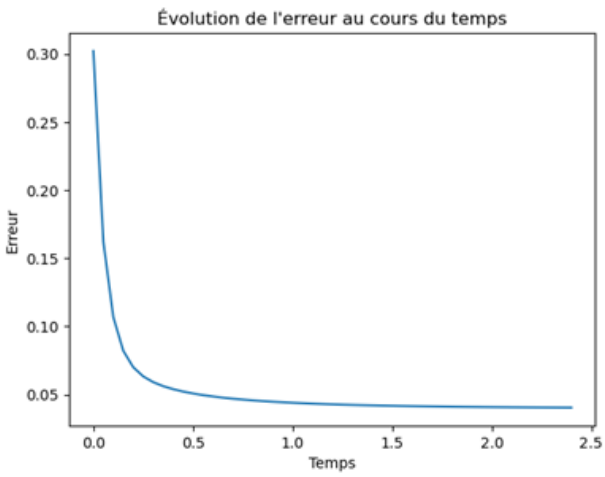

Figure 18: *Erreur en fonction du temps*

Le résultat montre clairement un déplacement du champ vers la droite, en plus de la diffusion, ce qui correspond à un écoulement dirigé. L'ajout systématique des courbes d'erreur à chaque étape a permis de valider numériquement les simulations et d'évaluer la stabilité et la précision du schéma. Ces résultats confirment que le modèle implémenté est fiable pour étudier l'évolution des interfaces dans les phénomènes de transition de phase.